

Reinforcement Learning - SS25

Niklas Rodenbüsch

September 7, 2025

1 Reinforcement Learning Fundamentals

Definition 1:

Core RL Components:

- **States** $s \in S$: Finite set of environment states
- **Actions** $a \in A$: Finite set of possible actions
- **Rewards** $r \in R$: Scalar feedback signals
- **Policies** π : Action selection strategies
 - Deterministic: $\pi(s) = a$
 - Stochastic: $\pi(a|s) = p(a|s)$
- **Value Function** $v^\pi(s) = \mathbb{E}[G_t | s_t = s]$: Expected return from state
- **Action-Value Function** $q^\pi(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a]$: Expected return from state-action pair

The goal in RL is to maximize expected cumulative reward. We often use **discounted rewards** to handle uncertainty about future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

where $\gamma \in [0, 1]$ is the discount factor.

Bellman Equations provide recursive relationships for value functions:

$$v^\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v^\pi(s')]$$

$$q^\pi(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma \sum_{a'} \pi(a'|s') q^\pi(s', a')]$$

2 Markov Decision Process (MDP)

Definition 2:

An MDP satisfies the **Markov Property**: the future depends only on the current state, not the history.

$$p(s', r | s_t, a_t) = p(s', r | s_t, a_t, \dots, s_0, a_0)$$

Example - Robot Cleaning: A robot has charge levels {high, low, none} and actions {explore, recharge}. The transition probabilities depend only on current charge and action, demonstrating the Markov property.

Optimal Policy π^* satisfies: $v^{\pi^*}(s) \geq v^\pi(s)$ for all states and policies.

3 Multi-Armed Bandits (MAB)

MAB problems have multiple actions but no states. The goal is to minimize **regret**:

$$\text{Regret} = \mathbb{E} \left[\sum_{t=1}^T (q^*(a^*) - q^*(a_t)) \right]$$

3.1 Solution Methods:

ϵ -Greedy: Choose best action with probability $1 - \epsilon$, random with ϵ . Action-value update: $q_{n+1} = q_n + \alpha(r_n - q_n)$

UCB (Upper Confidence Bound): Select action based on optimism in face of uncertainty:

$$a_t = \arg \max_a \left[q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

The second term promotes exploration of less-trying actions.

Softmax Selection: Choose actions probabilistically based on their values:

$$p(a_t = a) = \frac{\exp(q_t(a)/\tau)}{\sum_{a'} \exp(q_t(a')/\tau)}$$

where τ controls exploration ($\tau \rightarrow 0$: greedy, $\tau \rightarrow \infty$: random).

3.2 MAB Action Value Updates

Incremental update rule for action values:

$$q_{n+1}(a) = q_n(a) + \frac{1}{n} [r_n - q_n(a)]$$

Or with step size α :

$$q_{n+1}(a) = q_n(a) + \alpha [r_n - q_n(a)]$$

Where n is the number of times action a has been selected.

4 Dynamic Programming

Dynamic programming finds optimal policies using the Bellman equations when the MDP model is known.

Policy Iteration:

1. **Policy Evaluation:** Compute v^π for current policy
2. **Policy Improvement:** Update policy greedily: $\pi'(s) = \arg \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v^\pi(s')]$
3. Repeat until convergence

Value Iteration: Directly compute optimal value function:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v_k(s')]$$

Then extract optimal policy: $\pi(s) = \arg \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v^*(s')]$

4.1 DP Method Characteristics

Policy Iteration:

- Uses Bellman Expectation Equation for policy evaluation
- Guaranteed convergence to optimal policy
- Requires full model knowledge

Value Iteration:

- Uses Bellman Optimality Equation
- No explicit policy during iterations
- Extracts policy only at the end: $\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v^*(s')]$

5 Monte Carlo Methods

MC methods are **model-free** - they learn from experience without knowing the MDP model.

Key Characteristics:

- Update only after complete episodes
- Low bias (use actual returns)
- High variance (single sample estimates)
- No bootstrapping required

MC Prediction: Estimate value function by averaging returns:

$$v^\pi(s) = \frac{1}{N(s)} \sum_{i: s_t^{(i)}=s} G_t^{(i)}$$

MC Control Methods:

- **On-policy:** Use same policy for behavior and learning (e.g., e-greedy)
- **Off-policy:** Learn target policy from behavior policy using importance sampling

Importance Sampling for off-policy learning:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$$

This ratio corrects for the difference between target and behavior policies.

6 Temporal Difference Learning

TD methods combine advantages of DP (bootstrapping) and MC (model-free learning).

Key Advantages:

- Online learning (update after each step)
- Lower variance than MC
- Can handle infinite episodes

TD(0) Update:

$$v(s_t) \leftarrow v(s_t) + \alpha[r_{t+1} + \gamma v(s_{t+1}) - v(s_t)]$$

The **TD error** $\delta_t = r_{t+1} + \gamma v(s_{t+1}) - v(s_t)$ drives learning.

TD Control Methods:**SARSA (On-policy):**

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha[r_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)]$$

Q-Learning (Off-policy):

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a q(s_{t+1}, a) - q(s_t, a_t)]$$

Expected SARSA:

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha[r_{t+1} + \gamma \sum_a \pi(a|s_{t+1}) q(s_{t+1}, a) - q(s_t, a_t)]$$

Double Q-Learning addresses overestimation bias by maintaining two Q-functions:

$$q_1(s_t, a_t) \leftarrow q_1(s_t, a_t) + \alpha[r_{t+1} + \gamma q_2(s_{t+1}, \arg \max_a q_1(s_{t+1}, a)) - q_1(s_t, a_t)]$$

6.1 Monte Carlo vs Temporal Difference Comparison

Property	Monte Carlo	Temporal Difference
Variance	Higher	Lower
Bias	Unbiased (uses actual returns)	Biased (bootstrapping)
Initial value sensitivity	Less sensitive	More sensitive
Learning frequency	End of episode	Every time step
Episode requirement	Complete episodes	Can handle incomplete

7 Function Approximation

When state spaces are large or continuous, we need function approximation instead of tabular methods.

Definition 3:

Function Approximation: Represent value functions as parameterized functions $\hat{v}(s, \mathbf{w})$ where $\mathbf{w}$ are learnable parameters.

Linear Methods: $\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s)$ where $\mathbf{x}(s)$ is a feature vector. Linear TD methods have convergence guarantees.

Semi-gradient TD:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}_t) - \hat{v}(s_t, \mathbf{w}_t)] \nabla \hat{v}(s_t, \mathbf{w}_t)$$

Called "semi-gradient" because the target $r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}_t)$ also depends on $\mathbf{w}_t$.

Deep Q-Networks (DQN):

- Use neural networks to approximate Q-function
- **Experience Replay:** Store transitions, sample mini-batches randomly
- **Target Network:** Separate network for computing targets, updated periodically

Deadly Triad: Combining function approximation + bootstrapping + off-policy learning can cause instability.

Solutions:

- Target Networks (DQN): Separate network for computing targets
- Experience Replay: Break correlation in training data
- Double Q-Learning: Address overestimation bias

8 Policy Gradient Methods

Instead of learning value functions, directly optimize parameterized policies $\pi(a|s, \theta)$.

Theorem 1:

Policy Gradient Theorem: The gradient of expected return is:

$$\nabla J(\theta) = \mathbb{E}_{\pi}[q^{\pi}(s, a) \nabla \ln \pi(a|s, \theta)]$$

REINFORCE: Monte Carlo policy gradient method:

$$\theta_{t+1} = \theta_t + \alpha G_t \nabla \ln \pi(a_t|s_t, \theta_t)$$

High variance can be reduced using a **baseline** $b(s)$:

$$\theta_{t+1} = \theta_t + \alpha (G_t - b(s_t)) \nabla \ln \pi(a_t|s_t, \theta_t)$$

Actor-Critic Methods: Combine policy gradient (actor) with value function approximation (critic):

- **Actor:** Policy $\pi(a|s, \theta)$
- **Critic:** Value function $\hat{v}(s, \mathbf{w})$

Update using TD error: $\delta_t = r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}_t) - \hat{v}(s_t, \mathbf{w}_t)$

8.1 Score Function

For a policy $\pi(a|s, \theta)$, the score function is:

$$\nabla_{\theta} \ln \pi(a|s, \theta)$$

This appears in the policy gradient theorem and REINFORCE updates.

8.2 REINFORCE with Baseline

To reduce variance, subtract a baseline $b(s)$:

$$\theta_{t+1} = \theta_t + \alpha (G_t - b(s_t)) \nabla \ln \pi(a_t|s_t, \theta_t)$$

Common baselines: - State value function: $b(s) = v^{\pi}(s)$ - Average return - Any function that doesn't depend on the action

8.3 Deep Deterministic Policy Gradient (DDPG)

Classification: Off-policy method

Key features: - Uses experience replay buffer - Deterministic policy - Actor-critic architecture - Target networks for both actor and critic - Gaussian exploration noise added to actions

Why off-policy: - Experience replay uses transitions from previous policies - State distribution not generated by current target policy

9 Model-Based Reinforcement Learning

9.1 Overview

Learn a model f of the environment transition function $p(s', r|s, a)$, then use this model for planning.

9.2 Dyna Algorithm

Combines: - Direct RL: Learn from real experience - Planning: Learn from simulated experience using the model

Dyna-Q Steps:

1. Take action, observe reward and next state
2. Update Q-function with real experience
3. Update model with real experience
4. Repeat n times: sample from model and update Q-function

9.3 Model Learning

For Gaussian model: $f(s'|s, a; \theta) = \mathcal{N}(\mu(s, a; \theta), \Sigma)$

Where $\mu(s, a; \theta) = \theta^T \phi(s, a)$ with feature map $\phi(s, a)$.

9.4 Advantages and Disadvantages

Advantages: - More sample efficient - Can plan ahead - Can simulate experience

Disadvantages: - Model errors compound - Computationally expensive - Model bias affects performance

9.5 Planning Methods

Full Dynamic Programming: - Use learned model in Bellman equations - Computationally expensive for large state spaces

Sampling-based Planning: - Sample transitions from learned model - Apply MC or TD methods to sampled experience - More computationally efficient but higher variance

10 Advanced Topics

n-step Methods: Bridge between TD ($n = 1$) and MC ($n = \infty$):

$$G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n \hat{v}(s_{t+n})$$

TD(λ): Combines multiple n-step returns using eligibility traces:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$

Eligibility Traces: Track which states should receive credit for rewards:

$$\mathbf{z}_t = \gamma \lambda \mathbf{z}_{t-1} + \nabla \hat{v}(s_t, \mathbf{w}_t)$$

Model-Based RL:

- Learn environment model $p(s', r|s, a)$

- Use model for planning (generating simulated experience)
- **Dyna**: Interleave real experience with planning updates

Monte Carlo Tree Search (MCTS):

1. **Selection**: Navigate tree using tree policy
2. **Expansion**: Add new nodes
3. **Simulation**: Random rollout from leaf
4. **Backpropagation**: Update values along path

Key Tradeoffs:

- **Exploration vs Exploitation**: Balance trying new actions vs using known good ones
- **Bias vs Variance**: TD (higher bias, lower variance) vs MC (lower bias, higher variance)
- **Sample Efficiency vs Stability**: Off-policy (more efficient) vs on-policy (more stable)

11 Important Formulations for Exam

UCB Confidence Bound:

$$\sqrt{\frac{\ln t}{N_t(a)}}$$

TD Error:

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}) - v(s_t)$$

Importance Sampling Ratio:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$$

Semi-gradient Update:

$$w_{t+1} = w_t + \alpha[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla \hat{v}(s_t, w_t)$$

11.1 MDP Formalization Tips

When formalizing a problem as an MDP:

1. Clearly define state space S
2. Define action space A (may be state-dependent)
3. Specify reward function R
4. Define transition probabilities $p(s', r|s, a)$
5. Only include transitions with non-zero probability

11.2 Regret Analysis

Definition:

$$\text{Regret} = E \left[\sum_{t=1}^T (q^*(a^*) - q^*(a_t)) \right]$$

Asymptotic behavior: - ϵ -greedy: Linear regret (always explores with fixed probability) - UCB: Logarithmic regret (exploration decreases over time)